SHIFT CAMP | IMPLEMENTATION BLUEPRINT

Backend architecture and workflow guide

A practical plan for online registrations, on-arrival registration, attendee lookup, payments, and owner-managed admin access.

Recommended monorepo shape

Keep the public camper experience in **client/** and the trusted backend in **admin/shift-admin/**. The client calls the admin app API from server-side actions. This keeps database and payment secrets in one server deployment. Attendees do not need accounts; admins use an invite-only sign-in flow.

Area	Recommendation
Database	MongoDB Atlas with Mongoose schemas and indexes, accessed only by the admin app server.
Validation	Zod validates website and admin form data before backend business logic runs.
Attendee access	Pass ID + registration email, then an email verification code before personal details are returned.
Admin access	Clerk sign-in for admins only; you own the first account and invite or disable others.
Payment	Provider confirms payment server-to-server; a browser redirect never marks someone paid.

Three fields that must stay separate

Field	What it answers	Example
registrationSource	Where was the record created?	website or on_arrival
paymentMethod	How is the attendee paying?	card, bank_transfer, pay_on_arrival
paymentStatus	What is the payment state?	pending, due, paid, failed

A camper can register on the website and choose to pay on arrival. Registration source is not the same as payment method.

This is a recommended architecture, not an implemented backend. The registration UI is a frontend preview; database models, API credentials, and the payment provider still need to be selected and configured.

1. Registration workflows

Both entry paths create the same Registration document through the backend in **admin/shift-admin**. The source and creator fields distinguish how each record was created.

A. Website registration

- The camper completes the public form in **client/**. No attendee login is required.
- A client Server Action validates the fields with Zod, normalizes the email, and calls the backend API in **admin/shift-admin**.
- The admin app API validates the request and calls its server-side registration service. The service generates an Official Pass ID, sets **registrationSource = website**, and saves attendee, emergency contact, requests, and initial payment state.
- The site shows the pass ID and sends a confirmation email. Payment can be started separately when the camp fee and provider are ready.
- If online payment is selected, the backend creates a Payment record with the provider reference and marks it pending. A verified provider webhook later updates payment state.

B. On-arrival registration

- An authorized admin signs into **admin/shift-admin** and opens the New on-arrival registration form.
- The admin form sends the same required attendee fields to a protected admin action. Validate with the shared Zod schema plus any staff-only fields.
- The backend creates a pass ID and Registration document with **registrationSource = on_arrival** and **createdByAdminId** set to the signed-in admin.
- The admin records the accepted payment method. If paid at the desk, record the amount and payment type, then confirm it; otherwise leave payment due or pending.
- The camper receives or is shown their pass ID. It can be used by the same attendee lookup flow as a website registration.

Do not build a separate walk-in collection or pass-ID format. A single record shape makes attendee lists, exports, and payment reports consistent.

2. Check My Registration workflow

The pass ID is a reference, not a password. Require a second proof of access before exposing attendee details.

Collect pass ID and email	The page posts both values to a server action. Normalize and validate them; apply rate limits to avoid automated guessing.
Find the record	Query by the unique pass ID and matching normalized email. Return the same generic response for a non-match so the page does not reveal which identifiers exist.
Send a one-time code	Generate a short-lived code, store only its hash with an expiry and attempt count, and email the code to the matched registration email.
Verify before revealing	Check the code on the server, enforce expiry and attempt limits, then issue a short-lived lookup grant or fetch the limited ticket data.
Render the ticket	Show pass ID, camper, dates, venue, registration status, and payment badge. Do not include health notes or emergency contact on this public ticket.

Ticket state rules

Provider / method	Payment state	Ticket badge
Any method	paid	Paid and confirmed
bank_transfer	pending	Transfer pending
card	pending	Online payment pending
pay_on_arrival	due	Pay on arrival
No verified lookup yet	unknown	Status appears after lookup

The ticket can use the Official Pass ID visual from the success prototype. It does not need a QR code or coordinator panel.

3. Payment processing and states

Card payments and bank transfers can both be automated through a payment provider. Pay on arrival and any in-person payments are recorded by an admin. The website must not treat a return redirect, screenshot, or attendee claim as proof of payment.

Event	Server action	Stored state
Attendee chooses a provider method	Create a provider transaction from server code; save the provider reference and registration ID.	pending
Attendee returns to site	Show a pending screen and ask the server for current state. Do not mark paid from a query string.	pending until verified
Provider sends webhook	Verify the provider signature/reference, expected amount, currency, and registration mapping; process duplicate events idempotently.	paid or failed
Attendee pays at camp	Admin records the amount and accepted payment type, then confirms payment in the protected dashboard.	due -> paid

Suggested payment fields

Keep a current payment summary on the Registration for fast ticket and dashboard rendering. Store each provider attempt or payment event separately in Payments so retries, reversals, and refunds can be reconciled.

```
Registration.payment = {
  method: card | bank_transfer | pay_on_arrival | onsite,
  status: pending | due | paid | failed | refunded,
  currency: NGN,
  amountDueKobo: number, amountPaidKobo: number,
  provider: paystack | opay | null, providerReference: string | null,
  paidAt: Date | null
}
```

Payment collection is a separate decision from registration source. An online registrant may owe payment at camp; an on-arrival registrant may pay with whatever in-person methods the organizers accept.

The webhook Route Handler is public on the internet but must verify that each notification is authentic. Keep provider secret keys on the server. Use provider status verification when a callback is missing or ambiguous.

4. Admin access: you control invitations

There is no public admin sign-up form. Campers never create accounts. Admin identities are used only for the dashboard.

- Create your first owner identity in Clerk, then add its Clerk user ID to the AdminUser collection as role owner. Do this once through a protected setup script or manual bootstrap.
- The owner signs in at the admin sign-in route and opens Team management.
- An owner-only server action creates an invitation for the staff member's email. Clerk sends the invite link and handles admin account sign-in setup.
- After the invite is accepted, map the Clerk user ID to role admin in MongoDB. Admins do not receive owner privileges by default.
- For each protected read or write, resolve the session and check the active AdminUser role server-side. Check the same authorization in every Server Action and Route Handler that touches admin data.
- If a staff member leaves, the owner disables the AdminUser record and revokes or disables the identity provider account. Record major admin actions in AuditLog.

Suggested roles

Role	Typical permissions
owner	Invite and disable admins; view and manage all registrations; change system settings.
admin	Search attendees, record on-arrival registrations, update operational fields, and record payments.

Optionally add narrower roles later (for example, check-in only). Start with owner and admin so the first release remains manageable.

Protect at the data boundary

An admin layout or route redirect is helpful for user experience, but it is not the authorization boundary. Put reusable checks in a server-only data-access layer and call them from every sensitive page, Server Action, or API endpoint. Return only the fields each screen needs.

5. MongoDB collections and document shape

Use explicit core fields with optional nested details. MongoDB is flexible, but the app should still validate predictable fields and controlled status values.

Collection	Key fields	Purpose
registrations	passId, registrationSource, registrationStatus, attendee, emergencyContact, requests, paymentSummary, createdByAdminId, timestamps	One record for each camper from either entry path.
payments	registrationId, provider, providerReference, method, status, amountKobo, currency, verifiedAt, event IDs	Record payment attempts and provider confirmations for reconciliation.
adminUsers	authProviderUserId, email, role, active, createdAt	Maps Clerk identity to application role.
otpChallenges	registrationId, codeHash, expiresAt, attemptCount, consumedAt	Short-lived verification challenge; never store the raw email code.
auditLogs	actorAdminId, action, registrationId, timestamp, safe metadata	Track important staff actions without copying unnecessary sensitive data.

Registration document example

```
{
  passId: 'SHIFT-7K4P9D',                // unique, server-generated
  registrationSource: 'website',         // website | on_arrival
  registrationStatus: 'active',          // active | cancelled
  attendee: { fullName, phoneNumber, email, emailNormalized,
              fellowshipGroup, ageGroup },
  emergencyContact: { name, phone },
  requests: { dietaryAccessibility },    // optional, restricted
  payment: { method, status, currency, amountDueKobo, amountPaidKobo,
             provider, providerReference, paidAt },
  createdByAdminId: ObjectId | null,     // set for walk-ins
  createdAt: Date, updatedAt: Date
}
```

Useful indexes

- Unique index on passId so no two campers receive the same pass.
- Index on attendee.emailNormalized for pass-ID plus email matching; do not make email unique because families may share an address.
- Indexes on registrationSource, registrationStatus, and payment.status for admin filters and reporting.
- Unique index on provider + providerReference to prevent duplicate payment records.

Store amounts in kobo as integers. Restrict medical and emergency information to the staff who need it, and decide a retention/deletion policy before launch.

6. Monorepo file map

This project has two separate Next.js apps: the public camper experience in **client/** and the admin app in **admin/shift-admin/**. The paths below are proposed locations for the backend workflow, not files that already exist.

Public camper app - client/

Path	Function
client/app/register/actions.ts	Submit the public registration form to the backend API. It should not connect directly to MongoDB.
client/app/check-registration/actions.ts	Request and verify the email code through the backend API; receive only limited ticket details.
client/package.json	Public app dependencies such as Next.js and UI packages; no database secret belongs in this app.

Admin app - admin/shift-admin/

Path	Function
admin/shift-admin/app/sign-in/[[...sign-in]]/page.tsx	Admin sign-in page (invite-only identity provider flow).
admin/shift-admin/app/admin/layout.tsx	Admin layout and navigation; a convenience layer, not the authorization boundary.
admin/shift-admin/app/admin/page.tsx	Admin dashboard overview.
admin/shift-admin/app/admin/registrations/page.tsx	Search and filter camper records.
admin/shift-admin/app/admin/registrations/new/page.tsx	Create an on-arrival registration.
admin/shift-admin/app/admin/registrations/[id]/page.tsx	View an authorized attendee and payment details.
admin/shift-admin/app/admin/team/page.tsx	Owner-only admin invitation and access management.
admin/shift-admin/package.json	Admin app dependencies, including backend libraries used only by its server runtime.

Keep the admin dashboard and the backend in the admin app for now. The client talks to its API from server-side actions; it does not import admin source files or share MongoDB credentials.

Backend files and request flow

The admin app acts as the single trusted backend for both apps. This keeps MongoDB access, payment secrets, and admin authorization in one server deployment. Public API endpoints still need validation, abuse protection, and careful response shaping.

Backend routes and server modules - admin/shift-admin/

Path	Function
admin/shift-admin/app/api/registrations/route.ts	Public API to validate and create website registrations; generates the pass ID server-side.
admin/shift-admin/app/api/attendee-lookup/request-code/route.ts	Match pass ID and email, apply rate limits, and send an email verification code.
admin/shift-admin/app/api/attendee-lookup/verify-code/route.ts	Verify the code and return a minimal ticket response.
admin/shift-admin/app/api/webhooks/paystack/route.ts	Optional provider webhook; verify signature, amount, currency, and reference before updating payment.
admin/shift-admin/app/api/webhooks/opay/route.ts	Optional alternative provider webhook; only add after choosing OPay.
admin/shift-admin/app/admin/registrations/actions.ts	Admin-only management actions, including on-arrival creation and recording desk payments.
admin/shift-admin/lib/db/mongoose.ts	Connect to MongoDB Atlas and reuse the connection in this server app.
admin/shift-admin/lib/models/*.ts	Registration, payment, admin, OTP challenge, and audit log schemas and indexes.
admin/shift-admin/lib/auth/require-admin.ts	Check the signed-in identity maps to an active admin before sensitive actions.
admin/shift-admin/lib/services/registrations.ts	Shared creation logic for website and on-arrival records.
admin/shift-admin/lib/services/attendee-lookup.ts	Pass ID/email match, code lifecycle, attempt checks, and safe ticket response.
admin/shift-admin/lib/services/payments.ts	Create payment state and reconcile verified provider events idempotently.
admin/shift-admin/lib/validations/registration.ts	Zod schemas for public and staff registration payloads.

How the two apps use the backend

Workflow	Caller	Backend result
Website registration	client server action -> admin API	Create website registration and pass ID.
On-arrival registration	Admin UI -> protected admin action	Create the same record shape with source and staff ID.
Attendee lookup	client server action -> public API	Send OTP, then return a limited ticket after verification.
Payment confirmation	Provider -> signed webhook	Update payment only after trusted server verification.

Add admin/shift-admin as the only app with MONGODB_URI and payment-provider secrets. Set BACKEND_API_URL in the client deployment. Server-side client actions call that URL, so browser CORS is not needed; authenticate admin actions in the admin app itself. Add any new shared package to pnpm-workspace.yaml only if a real need appears.

7. Packages, configuration, and build order

Package / service	Use
mongoose	MongoDB models, validation, TypeScript inference, and indexes.
zod	Validate all incoming form and webhook payload shapes before business logic.
@clerk/nextjs	Admin-only sign-in and invitation flow; attendees remain account-free.
resend	Transactional email for confirmations and attendee verification codes.
@upstash/redis + @upstash/ratelimit (optional)	Shared rate limits for public OTP and lookup endpoints in serverless production.
Payment provider SDK/API (later)	Integrate only the selected provider; keep credentials server-side.

Environment variables

```
MONGODB_URI=...
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=...
CLERK_SECRET_KEY=...
RESEND_API_KEY=...
PAYMENT_PROVIDER_SECRET_KEY=...   # only after provider selection
PAYMENT_WEBHOOK_SECRET=...        # only after provider selection
```

Place secrets in local environment files ignored by Git and in the deployment provider's environment settings. Never prefix a secret with NEXT_PUBLIC_.

Step-by-step implementation sequence

01 Confirm required attendee fields, accepted payment methods, fee amount, and whether family members may share an email.

02 Create MongoDB Atlas project, database user with least privilege, network access rules, backups, and MONGODB_URI.

03 Add Mongoose connection and models; create pass ID and lookup indexes; validate requests with Zod.

04 Build the shared registration service, then connect the public register form and owner-protected on-arrival form.

05 Set up Clerk for admin-only identity; bootstrap your owner role; enforce authorization in server actions and DAL functions.

06 Add attendee lookup with email OTP, expiration, attempt limits, rate limiting, and a minimal ticket response.

07 Select a payment provider and complete merchant onboarding. Add payment initiation and signed webhook handling; test duplicate, late, failed, and reversed events.

08 Deploy with production secrets, review database access and backups, and test website and walk-in records through admin search and ticket lookup.

8. Security checklist and references

Before real attendee data is collected

- Only collect the health, accessibility, and emergency details organizers actually need; do not put these on the public ticket.
- Require email code verification before returning attendee details. Apply expiration, attempt caps, generic mismatch responses, and rate limits.
- Use unpredictable Pass IDs. A Pass ID is an identifier, not a secret or authorization token.
- Require an active admin role for every sensitive server read or write. The owner role alone can invite or disable admins.
- Verify payment notifications on the server against provider reference, expected amount, currency, and registration. Make webhook processing idempotent.
- Use least-privilege MongoDB credentials, TLS, backups, and a retention plan. Keep database and provider secrets out of client bundles and Git.
- Log sensitive admin changes but avoid writing medical values or full attendee records into logs.

Primary documentation

Next.js authentication, authorization, data-access layers, Server Actions, and Route Handlers:

https://nextjs.org/docs/app/guides/authentication

Clerk invite-only access and invitations:

https://clerk.com/docs/guides/secure/restricting-access
https://clerk.com/docs/guides/users/inviting

Mongoose TypeScript schemas:

https://mongoosejs.com/docs/8.x/docs/typescript/schemas.html

Resend with Next.js:

https://resend.com/nextjs

MongoDB Atlas security configuration:

https://www.mongodb.com/docs/atlas/setup-cluster-security/

Architecture summary

One Registration model; two registration sources; separate payment method and status; owner-controlled admin invitations; email OTP before attendee lookup; payment status changed only after trusted verification.